

1. What the system does

A sensor node inside the greenhouse reads temperature and humidity, packages them as JSON, and POSTs them over Wi-Fi to a Flask server on the local network. The server tags each reading with a zone and area, writes it to MongoDB Atlas, and a Streamlit dashboard reads from Atlas to show current conditions, historical charts and a raw data table. If a reading falls outside configured thresholds, the dashboard raises a flag and can send email alerts with a per-metric cooldown.

Advertised but not implemented: soil moisture, pH and light. These appear in the project docs, on the Home page feature cards, and as N/A metric tile on Zone 1, but no sensor, ingestion path or database field exists for any of them - do not focus on them. Same with Zone 2.

2. Architecture

2.1 The four tiers

Tier	Where	What it does
Edge	hardware/	MicroPython on a Pico W. Connects to Wi-Fi, reads a DHT sensor every few seconds, POSTs JSON to the ingestion server. A pure-Python mock sensor and sender let you run this tier on a laptop with no hardware.
Ingestion	hardware/pico_server.py	Flask on port 5000. Validates the payload, looks the node_id up in a hardcoded NODE_REGISTRY to resolve zone / area / target collection, and inserts into MongoDB. This is the path that actually feeds the dashboard.
Storage	MongoDB Atlas	Database greenhouse_db. Several collections with inconsistent schemas — see section 6.
Presentation	frontend/streamlit_app/	Streamlit multi-page app. Reads Atlas directly (not through an API), renders metric cards, Plotly charts and a raw table, evaluates alert thresholds and sends alert email.

The dashboard talks to MongoDB directly. There is no API layer between the frontend and the database, so the Streamlit process needs the Atlas connection string. backend/app/server.py

2.2 Diagrams

These two diagrams are from the repository. They describe the Pico W path (4.2, row 2) and are still accurate for that path.

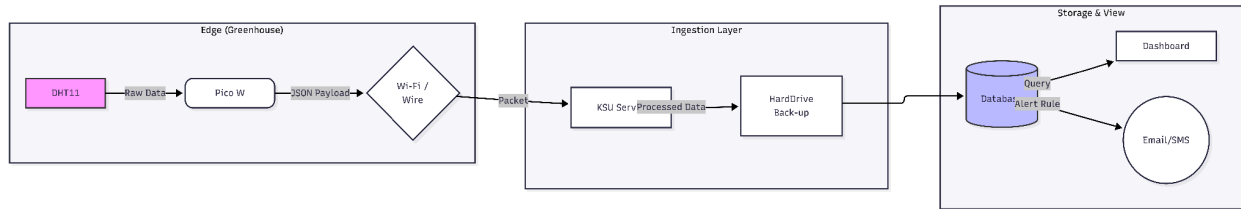


Figure 1 — End-to-end data flow. Note that "HardDrive Back-up" was planned, not built.

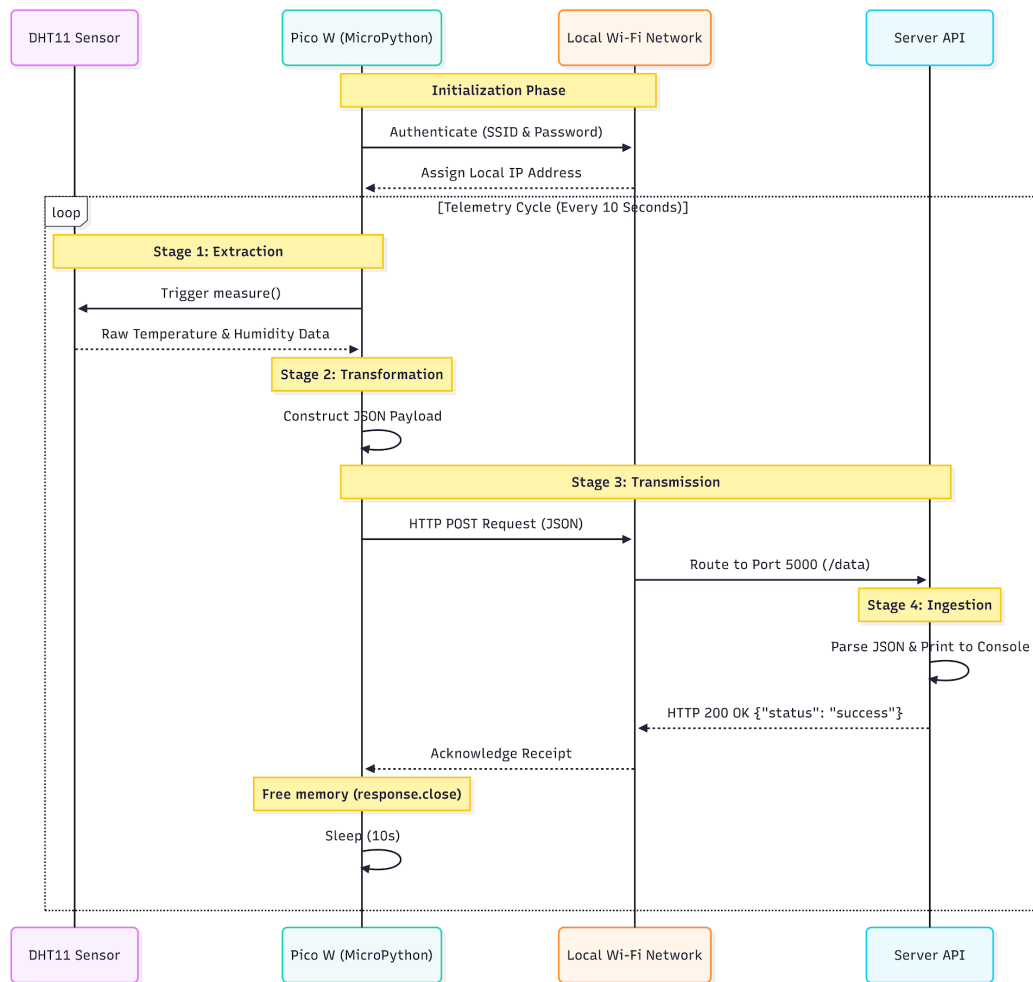


Figure 2 — Telemetry sequence, sensor node to ingestion server.

3. Repo map

hardware/ — edge and ingestion

File	Purpose
pico_server.py	The ingestion server that matters. Flask on 0.0.0.0:5000. Routes: GET /ping, GET /nodes, POST /data. NODE_REGISTRY maps node_id → zone / area / collection.
main.py	MicroPython for the Pico W: Wi-Fi connect, DHT22 read loop, POST every ~4s. Includes a triple-click BOOTSEL kill switch. Runs on the board, not on a PC.
mock_sensor.py	MockSensorNode class — generates plausible drifting temperature and humidity. Also contains a broken orphan function at module level (see 10.4).
mock_sender.py	Runs on a PC. Uses MockSensorNode to POST a fake reading to pico_server every 10s. This is how you demo the pipeline without hardware.

backend/ — the second, unused server

File	Purpose
app/server.py	Flask on 127.0.0.1:5050. POST /api/sensor-data writes to the sensor_data collection; GET /api/readings/latest reads it back. Nothing else uses either.
app/test/apirequest.py	The OpenWeather polling job. get_weather → transform_data → save_data, on a 20-minute schedule. Despite living under test/, this is production code — the root test suite imports it by file path.
app/test/main.py	Entry point for the polling job. Its import (from apirequest import ...) only resolves if you run it from inside that directory.
app/test/test_app.py	Three tests for server.py. Two of them hit the real Atlas cluster.
docs/schema.md, docs/performance.md	Short but accurate notes on the weather_data document shape and the ts index.

frontend/streamlit_app/ — the dashboard

File	Purpose
Home.py	Landing page. Three static feature cards and a greenhouse photo. Entry point: streamlit run Home.py.

File	Purpose
pages/l_zone1.py	The real dashboard. Data source resolution, temperature and humidity alerting with email dispatch, metric cards, three Plotly charts with a date-range filter, raw data table. Longest and most tangled file in the repo.
pages/settings.py	Threshold configuration per zone, unit toggle, email recipient management, live in/out-of-range preview.
utils/alerts.py	Threshold storage and evaluation, JSON persistence, per-metric email cooldown. The best-written module in the project.
utils/db.py	All MongoDB access. Six cached query functions — three for weather_data, three for zone1_dht22_test.
utils/layout.py	Rendering helpers. render_trend_chart and render_zone_health are never called.
utils/emailer.py	SMTP alert email. Clean and self-contained.
utils/config.py	Reads env vars for Mongo, OpenWeather and SMTP. CITY is hardcoded to "Akron,US".
utils/weather.py	Live OpenWeather call with a 5-minute cache. Requests imperial, converts to Celsius.
utils/themes.py, utils/styles.py, utils/sidebar.py	Three-theme colour dictionary, CSS loader, shared sidebar with zone nav and theme cycler.
utils/conversion.py	fahrenheitToCelsius, plus an inline assert-based self-test.
assets/style.css	Dashboard styling. Scoped under .stApp to limit the blast radius.
data/alert_settings.json	Persisted thresholds and email recipients, committed to the repo with real addresses in it.

4. Data model

Four document shapes for what is fundamentally the same pair of measurements. If you keep any of the year-one code, normalising this is the first thing worth doing.

zone1_dht22_test — the live sensor collection

```
{
  "node_id":    "pico_prototype_1_dht22",
  "zone":      "zone1",
  "area":      "upper_plants",
  "source":    "pico_w",
  "temperature_c": 22.4,
  "humidity_rh": 58.0,
  "ts":        ISODate("2026-04-13T20:11:02Z") // naive UTC
}
```

weather_data — the OpenWeather simulation collection - probably won't be needed

```
{
  "zone": "zone1", "area": "upper_plants",
  "source": "openweather", "sensor_type": "weather",
  "temp_c": 0.47,      // note: temp_c, not temperature_c
  "humidity_pct": 58,  // note: humidity_pct, not humidity_rh
  "city": "Akron", "temp_min": -0.48, "temp_max": 1.09,
  "weather_desc": "overcast clouds",
  "ts": ISODate("2026-03-18T23:41:24Z")
}
```

5. Running it without hardware

The mock sensor and mock sender let you simulate the whole pipeline i.e payload, validation, node routing, database insert, dashboard render on one laptop with no Pico W. This is the fastest way to see what the system actually does. You will need a MongoDB connection string; either stand up your own free Atlas cluster or a local mongod.

Setup

```
git clone https://github.com/Alpha-Project-Horticulture-Sensor/greenhouse-monitoring-system-year1.git
```

```
//Someone please check you can access and clone then get back to me
```

```
cd greenhouse-monitoring-system
python -m venv .venv
.venv\Scripts\Activate.ps1      # Windows PowerShell
source .venv/bin/activate       # macOS / Linux
pip install -r requirements.txt
```

Run — three terminals

1. **Ingestion server:** `cd hardware && python pico_server.py` — prints a banner and confirms the database connection.
2. **Mock sender:** `cd hardware && python mock_sender.py` — posts a synthetic reading every 10 seconds. Watch the server terminal log each insert.
3. **Dashboard:** `cd frontend/streamlit_app && streamlit run Home.py` — opens on `http://localhost:8501`. Click Zone 1 in the sidebar.

6. What is worth keeping (Maybe perhaps?)

- **The `NODE_REGISTRY` pattern in `pico_server.py`** — mapping an incoming `node_id` to its zone, area and destination collection, and rejecting anything unregistered with a 403. This is the right shape for a multi-sensor greenhouse and generalises directly to Shelly devices.
- **The mock sensor and sender pair** — being able to demo the full pipeline with no hardware is genuinely useful, and it is how I would suggest you develop against the dashboard. Keep the idea even if you rewrite the code.
- **`utils/emailer.py`** — one generic sender with thin temperature and humidity wrappers, returning (ok, message) tuples instead of raising. Easy to test, easy to port.
- **The caching approach in `utils/db.py`** — `@st.cache_resource` for the client and `@st.cache_data(ttl=...)` per query, with short TTLs for live values and longer ones for history. Correct use of Streamlit's two cache types.

7. First Week

First-week checklist

1. Figure out Atlas MongoDB, set up your own cluster.
2. Clone, install, and get the mock pipeline running
3. Read through the modules, hardware and frontend are the two worth understanding.
4. Research Shelly H&T and find out how it actually reports, then shape your ingestion tier.
5. Agree on a single document schema and write it down before anyone writes ingestion code.